
Project 2: Kernel Thread in xv6

Adapting Linux's clone() model

Due date
2026. 05. 27. 23:59

Overview

- **Goal**

- This project involves implementing a simplified version of **kernel-level threads** in the xv6 operating system.

- **Submission Guidelines**

- Submit the modified **xv6 source code** and the **report** for this assignment.
- Push all your source code and your report to the following GitHub Classroom repository's **main branch**:
 - https://classroom.github.com/a/35mf_CDQ

- **Deadline: May 27, 2026 (Wed) 23:59**

- Late submissions are allowed, but your final score will be reduced by **5%** for each **3-hour window** following the due date.
- Grading is based on your **final commit**. Be extremely cautious about committing after the deadline, as it will trigger the late penalty.

- **Short quiz** in deadline week - held in your regular class **(Thu May 28 / Fri May 29)**

Contents

- The Idea
 - Process vs thread, Linux's clone() insight
- The Design
 - mm_struct, refcounts, the API
- Your Work
 - Three steps to implement kernel-thread
 - Things to keep in mind

Part 1: The Idea

From a question to a design choice

The big question

- What is the difference between a process and a thread?
- You may say...
 - "threads are lighter", "they share memory", "many threads in one process"
 - All true, but vague.
- By the end of today you should be able to answer:

"Process and thread are **the same kernel object**. Only what they **share differs**."

Why threads at all

- **Parallelism** – use multiple cores
- **Hide blocking** – keep working while one path waits on I/O
- **Cheap sharing** – share common data without IPC
- **Cheap context switch** – use same page table
- **Need: execution units that share an address space**

xv6: Everything in one struct

- In xv6, struct proc bundles execution (context, kstack) with address space (pagetable, sz) in one struct.
- They share one lifecycle - Allocated, freed, and switched together
- So a second execution unit(= a thread) requires a second address space.
- To enable threads, this bundle must be broken apart.

```
$ vim kernel/proc.h

struct proc {
    struct context context;           // execution
    pagetable_t    pagetable;         // address space
    uint64         sz;                // memory size
    struct file    *ofile[NOFILE]    // files
}
```

Linux's insight: split them

- Linux splits the task representation into two distinct entities.
 - **task_struct**: Execution unit (one per task).
 - **mm_struct**: Memory management unit (referenced via pointer).

```
struct task_struct {  
    ...  
    struct mm_struct *mm;  
    ...  
};
```

```
struct mm_struct {  
    pgd_t *pgd;  
    atomic_t mm_users;  
    // ~100 fields...  
};
```

- Two tasks can point the same mm_struct (= two threads on the same process)

One system call, two meanings

- The kernel only provides `clone()` system call.
- `fork()` is a library function.
- The `CLONE_VM` bit decides whether the address space is *shared or copied*.

```
fork()           == clone(SIGCHLD)
pthread_create() == clone(CLONE_VM | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD...)
```

Project goal in one diagram

- Same system call, Different flag bit.

```
graph TD; A["fork() == clone(0) [user/ulib.c - given]  
thread_create() == clone(CLONE_VM) [user/uthread.c - yours]"] --> B["one sys_clone() in the kernel"]; B --> C["if CLONE_VM → create new thread  
else → fork new process"]
```

fork() == clone(0) [user/ulib.c - given]
thread_create() == clone(CLONE_VM) [user/uthread.c - yours]

one sys_clone() in the kernel

if CLONE_VM → create new thread
else → fork new process

Part 2: The Design

Structures, refcounts, and the API

Our mm_struct

- Linux has ~100 fields here. We keep only four.
- The core idea remains the same: separate shared resources from per-task state.

```
$ vim kernel/proc.h

struct mm_struct {
    pagetable_t    pagetable;
    uint64         sz;
    int            refcount;    // how many tasks share this mm
    struct spinlock lock;
}
```

Reference Counting

- The last one out cleans up
- Every `mm_get()` must be matched by exactly one `mm_put()`

```
mm_alloc()    →  ref_count = 1;
mm_get()      →  ref_count++;   (a new task starts sharing)
mm_put()      →  ref_count--;   (a task lets go)

                →  if refcount==0, free pagetable + free mm
```

struct proc

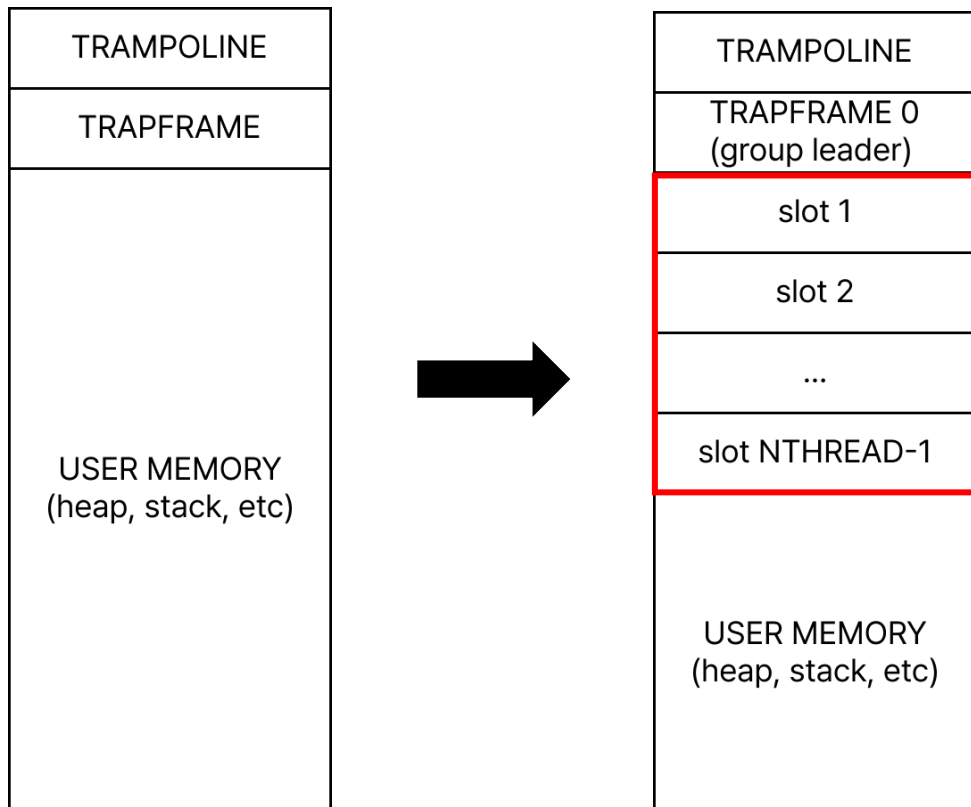
- Redesign struct proc to decouple shared resources.
 - Per-task resources: kstack, trapframe, context, mm pointer
 - Shared resources within a process: pagetable, sz (encapsulated in mm)
- Expend per-task states for thread execution (i.e. group_leader, tgid, ...)

```
$ vim kernel/proc.h

struct proc {
    struct mm_struct *mm;           // replaces pagetable + sz
    struct proc *group_leader;      // who leads this thread group
    int tgid;                       // = group_leader->pid
    uint64 tf_va;                   // where my trapframe is mapped
    void *ustack;                   // user stack base (for join)
}
```

One page table, many trapframes

- Threads share a pagetable – but each needs its own trapframe.
- Allocate distinct physical memory pages and map them to unique VAs.



```
#define NTHREAD 8
#define TRAPFRAME (TRAMPOLINE - PGSIZE)
#define USERTOP (TRAPFRAME - NTHREAD * PGSIZE)
```

New function: `find_free_tf_va()`
- returns an unused slot's VA, or 0 if all taken.

FYI: How threads find their own trapframe

- Each thread needs to find *its* trapframe on every trap
 - Stash the VA in the `sscratch` CSR before returning to user.
- Kernel → User(prepare_return): sets `sscratch` to `p->tf_va`
- User → Kernel(uservec):
 - one instruction – “`csrrw a0, sscratch, a0`” swaps `a0` with `sscratch`.
 - Now `a0` = TRAPFRAME VA, `sscratch` = saved user `a0`
- Same trick as Linux's `tp` (thread pointer)

New System Calls and User APIs

Kernel system call Interface

```
$ vim user/user.h
```

```
int clone(void (*fn)(void *), void *arg, void *stack, int n_pages, int flags);  
int join(void **stack);
```

User-space library API

```
$ vim user/uthread.h
```

```
int thread_create(void (*fn)(void *), void *arg, int n_pages);  
int thread_join(void);
```

- arg – single opaque pointer, like pthread_create. Pack multiple values via a struct
- n_pages – size of the user stack, in pages. The kernel uses it to set the new thread's sp
- The library (thread_create) malloc's (n_pages * PGSIZE); The kernel does not allocate the stack.

What we already did, so you don't have to...

- You will only modify a small set of functions inside these files
 - kernel/proc.c, kernel/exec.c, user/uthread.c
- Don't modify
 - kernel/proc.h → struct definitions (struct mm_struct, struct proc, CLONE_VM)
 - kernel/param.h, kernel/memlayout.h → (NTHREAD, USERTOP)
 - kernel/trampoline.S, kernel/trap.c → sscratch-based trap path
 - kernel/syscall.h, kernel/sysproc.c, kernel/usys.pl
 - user/ulib.c → fork() wrapper
- You may add helper functions inside kernel/proc.c or user/uthread.c if it helps
- See [git diff](#) against the original xv6 for what changed

Part 3: Your Work

From skeleton to working threads

The three steps

- Step 1. Extend memory management to support shared address spaces
- Step 2. Implement core threading system calls and user-level wrappers
- Step 3. Coordinate process-wide system calls' behaviors (exec, exit, wait, kill)
- About ~125 LoC total.
- **Sketch the flow and the big picture first. Diving straight into code will hurt.**

Boot sanity check

- Right after you clone the starter,

```
$ make qemu CPUS=1  
  
init: starting sh  
$ls                      # works normally  
$usertests forktest  
test forktest: PASS  
...  
FAILED -- lost some free pages **(out of **)
```

- xv6 boots, sh works, fork works.
- **Fork-exit cycles leak!! → Step 1 clue**

Step1: Extend Memory Management

- Implement:
 - `mm_get()`, `mm_put()`
 - `find_free_tf_va()`
 - per-thread trapframe slot unmap inside `freeproc()`
- Done when usertests passes with no leak message.

Step2: Implement Core Threading Primitives

- Implement:
 - CLONE_VM branch of kclone() (kernel/proc.c)
 - kjoin() (kernel/proc.c)
 - thread_create() / thread_join() (user/uthread.c)
- Read first: the fork branch of kclone() already works.
- Compare: what is *shared* vs *newly allocated*?
- Stack lifetime is the library's job:
 - thread_create : allocate n_pages stack using malloc()
 - thread_join : free stack returned by join() system call using free()

Step3: Coordinate Process-wide System Calls

- Update:
 - kwait – only count children that are group leaders
 - kexit
 - when called by leader: kill and synchronously reap siblings
 - when called by a thread: only this task exits, siblings continue
 - kkill – kill the entire group, not just one task
 - kexec – drain siblings before swapping the page table
 - exec from non-leader is banned in our implementation
- Note: this is a simplified take on Linux's group teardown

Things to keep in mind

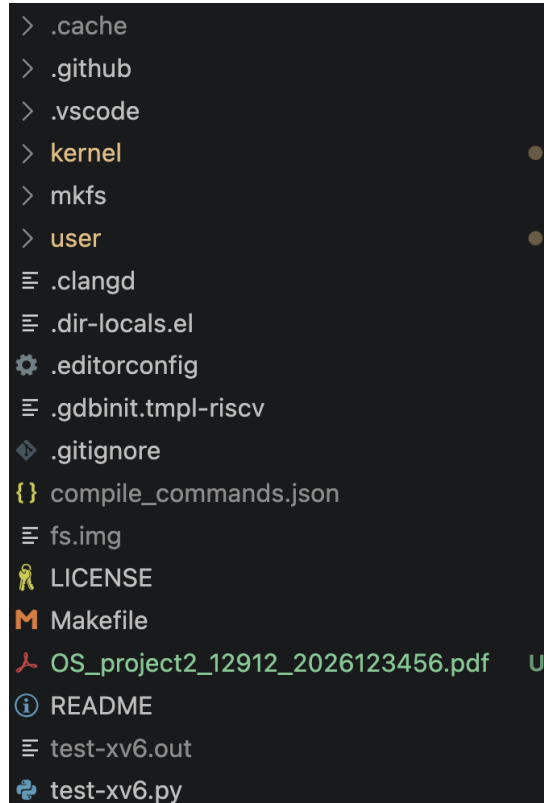
- While implementing, watch out for these design choices:
- `getpid()` returns per-task PID
 - In our model, each thread has its own PID.
 - `getpid()` called inside a thread returns a different value than the leader's PID.
 - Be intentional when your test code uses `getpid()`.
- File descriptors and `cwd` are always duplicated, never shared
 - `thread_create()` dups the parent's open files and `cwd`.
 - Threads do NOT share fd table (unlike POSIX threads).
 - Reading/Writing the same file from two threads operates on independent file structs.

Report

- **Code Analysis**
 - Identify what has been added to the provided code and analyze the purpose of each addition.
- **Design**
 - Explain your implementation plan to meet the assignment requirements.
- **Implementation**
 - Detail the specific modifications and additions you made to the given xv6 codebase. Clearly highlight how they differ from the original code and the purpose of these changes.
- **Results**
 - Document the compilation and execution process. Include screenshots verifying that all requirements are met, alongside explanations of the execution flow.
- **Troubleshooting**
 - If you encountered any issues during the assignment, describe the issues and your resolution process. For unresolved issues, explain what they were and what approaches you attempted.

Report

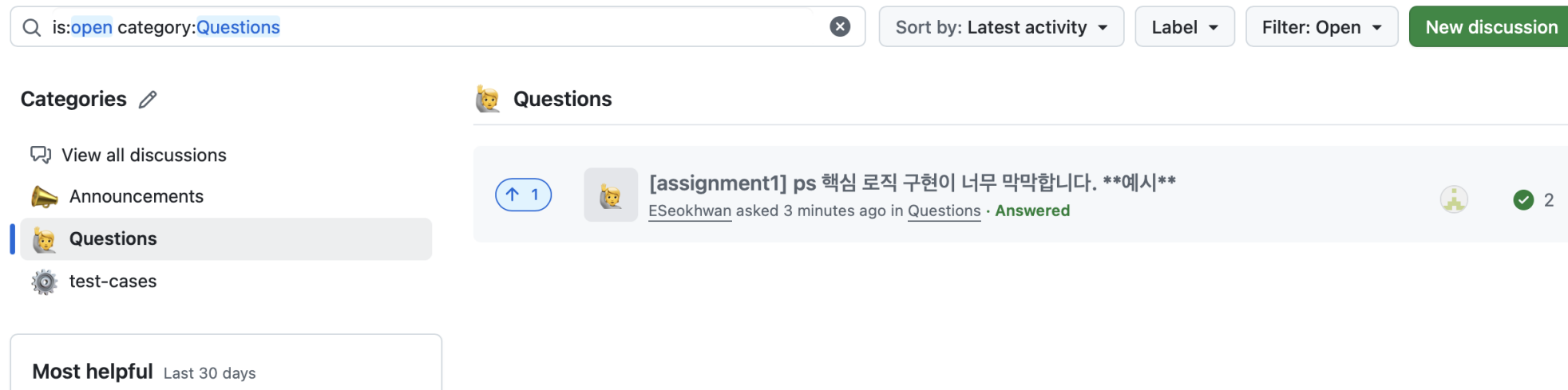
- The report must be a PDF file strictly named: **OS_project2_[class number]_[student ID].pdf**
- You must commit and push your report PDF to your GitHub Classroom repository along with your xv6 source code.
- Make sure that report PDF is in the root directory of your repository, as shown in below.



```
> .cache
> .github
> .vscode
> kernel
> mkfs
> user
≡ .clangd
≡ .dir-locals.el
⚙ .editorconfig
≡ .gdbinit.tmpl-riscv
💎 .gitignore
{} compile_commands.json
≡ fs.img
🔑 LICENSE
📄 Makefile
📄 OS_project2_12912_2026123456.pdf
📖 README
≡ test-xv6.out
🔗 test-xv6.py
```

Q&A

- For assignment related questions, please use the **Discussions tab** of xv6-sandbox repository.
 - <https://github.com/2026-HYU-ELE3021/xv6-riscv-sandbox/discussions>
- **Questions sent by email will not be answered.**
- For non-assignment questions or personal matters, please use an email like before.



- Students are highly **encouraged to answer each other's questions!** Active participation is greatly appreciated and helpful for everyone

Test Case Contribution

- You can share your custom test cases on the **Discussions tab** of xv6-sandbox repository too.
 - <https://github.com/2026-HYU-ELE3021/xv6-riscv-sandbox/discussions>
- A structured template is already provided, so we highly encourage your active contributions.
- Well-crafted test cases might actually be adopted for the official grading process!

Sort by: Latest activity ▾ Label ▾ Filter: Open ▾ [New discussion](#)

Q is:open category:test-cases

Categories

- View all discussions
- Announcements
- Questions
- test-cases**

test-cases

1 **[lab03] getppid usertests**
wnsah814 started 1 hour ago in test-cases 0

Most helpful Last 30 days

2026-HYU-ELE3021 / xv6-riscv-sandbox

Code Pull requests **Discussions** Actions Projects Wiki Security Insights Settings

Start a new discussion

test-cases

If this doesn't look right you can [choose a different category](#).

Add a title

[과제 이름{assignment1, proj1, proj2, ...}] <테스트할 기능이나 버그를 요약해주세요>

1. 테스트 목적 (Objective)

이 테스트 케이스가 무엇을 검증하기 위한 것인지 간략하게 설명해 주세요.

예) 이 테스트는 파이프가 꽂 찾을 때 부모와 자식 간의 데드락이 발생하는지 검증합니다.

2. 테스트 코드 (Test Code)

user/usertest.c 에 추가하거나 독립적인 유저 프로그램으로 실행할 C 코드를 작성해 주세요.

```
```c
int
my_custom_test(char *s)
{
```

**3. 실행 방법 (How to Run)**

다른 학우들이 이 테스트를 어떻게 실행해 볼 수 있는지 명령어를 적어주세요.

1. user/usertest.c 에 테스트 코드 함수를 붙여넣고...
2. make qemu 후 \$ usertests custom\_test...